

Java 실무형 코딩 테스트 (중고차 수출 플랫폼)

J-01. 컬렉션 — 브랜드별 차량 목록 분류

문제

```
List<String> cars = List.of(
    "Toyota|Camry|2020|15000",
    "Honda|Civic|2019|12000",
    "Toyota|Corolla|2021|17000",
    "Honda|Accord|2022|22000",
    "BMW|3Series|2020|35000"
);
```

- 문자열을 파싱하여 브랜드(brand)를 key로, 차량 모델명 List를 value로 갖는 `Map<String, List<String>>` 을 생성하라
- 결과를 출력할 때 브랜드 이름을 알파벳 순으로 정렬하여 출력하라
- Collections 또는 Map 관련 메서드만 사용할 것 (Stream 금지)
- 출력 예시:

```
BMW: [3Series]
Honda: [Civic, Accord]
Toyota: [Camry, Corolla]
```

정답 코드

```
Map<String, List<String>> result = new TreeMap<>();

for (String car : cars) {
    String[] parts = car.split("\\|");
    String brand = parts[0].trim();
    String model = parts[1].trim();
    result.computeIfAbsent(brand, k -> new ArrayList<>()).add(model);
}

result.forEach((brand, models) ->
    System.out.println(brand + ": " + models));
```

출제 의도

실무에서 가장 흔한 작업인 "목록을 key-value 구조로 분류"하는 능력을 본다. API 응답 데이터나 DB 결과를 그룹핑하는 패턴과 동일하다. Stream을 금지하여 기본 Map/List 조작 능력을 검증한다.

핵심 포인트

- `computeIfAbsent`로 key 없을 때 List를 자동 초기화
- `TreeMap` 사용 시 자동 알파벳 정렬이 되므로 별도 정렬 불필요

합정

- `split("|")`으로 쓰면 정규식 OR 연산자로 해석되어 모든 문자가 분리된다
- 반드시 `split("\\|")`로 이스케이프해야 한다

J-02. 컬렉션 — 중복 차량 번호 탐지

문제

```
List<String> vinNumbers = new ArrayList<>(Arrays.asList(
    "VIN001", "VIN002", "VIN003", "VIN001",
    "VIN004", "VIN002", "VIN005"
));
```

1. 중복된 VIN 번호만 추출하여 `Set<String>`에 담아라
2. 원본 리스트에서 중복 항목을 제거한 고유 목록을 만들어라 (순서 유지)
3. "중복 발견: X건" 형태로 중복 수를 출력하라
4. `Set`, `LinkedHashSet` 등 컬렉션 특성을 활용할 것

정답 코드

```
Set<String> seen = new HashSet<>();
Set<String> duplicates = new LinkedHashSet<>();

for (String vin : vinNumbers) {
    if (!seen.add(vin)) {
        duplicates.add(vin);
    }
}

List<String> unique = new ArrayList<>(new LinkedHashSet<>(vinNumbers));

System.out.println("중복 발견: " + duplicates.size() + "건");
System.out.println("중복 VIN: " + duplicates);
System.out.println("고유 목록: " + unique);
```

출제 의도

`Set`의 `contains/add` 반환값 활용 능력과 `LinkedHashSet`의 순서 보장 특성을 아는지 본다. 실무에서 중복 데이터 검증은 매우 빈번한 작업이다.

핵심 포인트

- `Set.add()`의 반환값 `false`를 활용해 중복 탐지
- `LinkedHashSet`으로 삽입 순서 유지

합정

- `HashSet`으로 변환하면 순서가 보장되지 않는다
- `LinkedHashSet`을 활용해야 삽입 순서를 유지하며 중복을 제거할 수 있다

J-03. Stream API — 수출 가능 차량 필터링

문제

```
class Car {
    String model;
    int year;
    int price;
    boolean exported;

    Car(String model, int year, int price, boolean exported) {
        this.model = model; this.year = year;
        this.price = price; this.exported = exported;
    }
    // getter 생략
}

List<Car> inventory = List.of(
    new Car("Camry", 2020, 15000, false),
    new Car("Civic", 2017, 8000, false),
    new Car("Corolla", 2021, 17000, true),
    new Car("Accord", 2019, 11000, false),
    new Car("X5", 2022, 45000, false),
    new Car("3Series", 2016, 7000, false)
);
```

1. `exported`가 `false`인 차량만 추출하라
2. 연식이 2018년 이후(포함)인 차량만 남겨라
3. 가격 오름차순으로 정렬하라
4. 모델명과 가격만 "모델명: \$가격" 형태로 출력하라
5. Stream API(`filter`, `sorted`, `map`, `forEach`)를 사용할 것

정답 코드

```
inventory.stream()
    .filter(car -> !car.isExported())
    .filter(car -> car.getYear() >= 2018)
    .sorted(Comparator.comparingInt(Car::getPrice))
    .map(car -> car.getModel() + ": $" + car.getPrice())
    .forEach(System.out::println);
```

```
// 출력:
// Accord: $11000
// Camry: $15000
```

출제 의도

Stream의 `filter` → `sorted` → `map` → `forEach` 파이프라인을 이해하는지 본다. 실무에서 가장 많이 쓰는 Stream 패턴이다.

핵심 포인트

- 복수 `filter` 조건을 체이닝하거나 단일 `filter`에 `&&`로 결합 모두 가능
- `Comparator.comparingInt` 활용으로 간결하게 정렬

함정

- `sorted()` 인자 없이 호출하면 `Comparable`이 구현되지 않아 런타임 에러 발생
- `map`은 중간 연산이므로 반드시 `forEach` 등 최종 연산으로 마무리해야 한다

J-04. Stream API — 연도별 평균 가격 집계

문제

J-03의 `Car` 클래스와 `inventory` 데이터를 그대로 사용한다.

1. 전체 차량을 연식(`year`) 기준으로 그룹핑하라
2. 각 연식 그룹의 평균 가격을 계산하라
3. 연식 오름차순으로 정렬하여 출력하라
4. 출력 형식: "2016년: \$7000.00"
5. `Collectors.groupingBy`와 `Collectors.averagingInt`를 활용할 것

정답 코드

```
Map<Integer, Double> avgByYear = inventory.stream()
    .collect(Collectors.groupingBy(
        Car::getYear,
        Collectors.averagingInt(Car::getPrice)
    ));

avgByYear.entrySet().stream()
    .sorted(Map.Entry.comparingByKey())
    .forEach(e ->
        System.out.printf("%d년: %.2f\n", e.getKey(), e.getValue()));
```

출제 의도

`Collectors.groupingBy` + downstream collector 조합을 알고 있는지 확인한다. SQL의 `GROUP BY AVG()`를 Java 코드로 구현하는 실무 능력을 본다.

핵심 포인트

- `groupingBy` + `averagingInt` 조합
- 결과 Map을 정렬하려면 `entrySet().stream()`으로 재처리해야 한다

함정

- `groupingBy` 결과는 `HashMap`이므로 순서가 보장되지 않는다
- `Map.Entry.comparingByKey()`로 스트림 재처리해야 정렬된 출력이 가능하다

J-05. 문자열 처리 — 차량 정보 파싱 및 마스킹

문제

```
String raw = "  toyota camry | 2020 | owner:kim cheol-su | price:15000usd
";
```

1. 앞뒤 공백을 제거하라
2. `|`를 구분자로 분리하여 각 항목의 앞뒤 공백도 제거하라
3. 차량 모델명의 첫 글자를 대문자로 변환하라 (예: `toyota camry` → `Toyota Camry`)
4. `owner` 항목에서 이름의 성(첫 단어)만 남기고 이름은 `***`로 마스킹하라 (예: `kim cheol-su` → `kim ***`)
5. `price`에서 숫자만 추출하여 정수로 변환하라
6. 최종 출력: `"Toyota Camry / 2020 / kim *** / 15000"`

정답 코드

```
String trimmed = raw.trim();
String[] parts = trimmed.split("\\|");

// 모델명 Title Case
String[] modelWords = parts[0].trim().split(" ");
StringBuilder model = new StringBuilder();
for (String w : modelWords) {
    model.append(Character.toUpperCase(w.charAt(0)))
          .append(w.substring(1).toLowerCase())
          .append(" ");
}
String modelName = model.toString().trim();

// 연식
String year = parts[1].trim();

// 이름 마스킹
String ownerPart = parts[2].trim().replace("owner:", "");
String lastName = ownerPart.split(" ")[0];
```

```
String masked = lastName + " ***";

// 가격 숫자 추출
String pricePart = parts[3].trim().replace("price:", "").replaceAll("[^0-9]", "");

System.out.println(modelName + " / " + year + " / " + masked + " / " + pricePart);
```

출제 의도

`trim`, `split`, `substring`, `replace` 등 문자열 메서드의 실전 활용과 개인정보 마스킹 처리 능력을 본다. 실무 API 연동 시 외부 데이터를 가공하는 매우 흔한 패턴이다.

핵심 포인트

- `replaceAll("[^0-9]", "")`으로 숫자 외 문자를 전부 제거
- Title Case 변환은 단어 단위로 `charAt(0)` → 대문자, `substring(1)` → 소문자 처리

함정

- `split(" ")`으로 나눌 때 연속 공백이 있으면 빈 문자열이 포함될 수 있다
- 모델명 단어가 이미 대문자인 경우 `toLowerCase()` 없이 `toUpperCase()`만 쓰면 나머지 글자가 그대로 유지된다

J-06. 문자열 처리 — 차량 검색 쿼리 정규화

문제

```
String userInput = " TOYOTA camry 2020 ";
```

1. 앞뒤 공백 제거 후 연속된 공백을 단일 공백으로 정규화하라
2. 모든 단어의 첫 글자만 대문자, 나머지는 소문자로 변환하라 (Title Case)
3. 각 단어를 "-"로 연결한 검색 slug를 만들어라 (예: `Toyota-Camry-2020`)
4. `StringBuilder`를 사용하여 slug를 생성할 것
5. `String.split()`, `String.toLowerCase()`, `String.toUpperCase()` 활용

정답 코드

```
String normalized = userInput.trim().replaceAll("\\s+", " ");
String[] words = normalized.split(" ");

StringBuilder slug = new StringBuilder();
for (int i = 0; i < words.length; i++) {
    String word = words[i];
    String titled = Character.toUpperCase(word.charAt(0))
```

```

        + word.substring(1).toLowerCase();
    slug.append(titled);
    if (i < words.length - 1) slug.append("-");
}

System.out.println(slug.toString()); // Toyota-Camry-2020

```

출제 의도

정규식 기반 공백 정규화 및 Title Case 변환, `StringBuilder` 활용 능력을 본다. 검색 쿼리 전처리나 URL slug 생성은 실무에서 자주 등장하는 패턴이다.

핵심 포인트

- `replaceAll("\\s+", " ")`로 다중 공백을 단일 공백으로 정규화
- `StringBuilder`로 반복 문자열 연결 시 + 연산보다 성능적으로 유리

함정

- `trim()` 없이 `split(" ")`하면 앞뒤 공백이 빈 문자열 원소로 들어온다
- `word.substring(1).toLowerCase()` 없이 대문자 변환만 하면 "CAMRY" → "CAMRY"로 그대로 나온다

J-07. 객체 정렬 — 다중 조건 정렬

문제

```

class ExportCar {
    String brand;
    String model;
    int year;
    int price;

    ExportCar(String brand, String model, int year, int price) {
        this.brand = brand; this.model = model;
        this.year = year; this.price = price;
    }
    // toString, getter 구현 가정
}

List<ExportCar> cars = new ArrayList<>(List.of(
    new ExportCar("Toyota", "Camry", 2020, 15000),
    new ExportCar("Honda", "Civic", 2020, 12000),
    new ExportCar("Toyota", "Corolla", 2019, 11000),
    new ExportCar("Honda", "Accord", 2021, 22000),
    new ExportCar("Toyota", "Yaris", 2019, 9000)
));

```

1. 브랜드(brand) 알파벳 오름차순으로 먼저 정렬하라

2. 같은 브랜드 내에서는 연식(year) 내림차순으로 정렬하라
3. 연식도 같다면 가격(price) 오름차순으로 정렬하라
4. `Comparator.comparing().thenComparing()` 체이닝 방식을 사용할 것
5. 정렬 결과를 출력하라

정답 코드

```
cars.sort(
    Comparator.comparing(ExportCar::getBrand)

    .thenComparing(Comparator.comparingInt(ExportCar::getYear).reversed())
    .thenComparingInt(ExportCar::getPrice)
);

cars.forEach(System.out::println);

// 출력 순서:
// Honda / Accord / 2021 / 22000
// Honda / Civic / 2020 / 12000
// Toyota / Camry / 2020 / 15000
// Toyota / Corolla / 2019 / 11000
// Toyota / Yaris / 2019 / 9000
```

출제 의도

`Comparator` 체이닝으로 다중 정렬 기준을 구현하는 능력을 본다. 실무에서 목록 정렬은 항상 복합 조건이 들어오기 때문에 필수 패턴이다.

핵심 포인트

- `thenComparing`으로 보조 정렬 기준 추가
- 내림차순이 필요한 항목에만 `.reversed()` 적용

함정

- `thenComparingInt(ExportCar::getYear).reversed()`는 전체 `Comparator`를 뒤집으므로 의도와 다른 결과가 나온다
- 내림차순이 필요한 항목은 `Comparator.comparingInt(...).reversed()`를 별도로 만들어 `thenComparing`에 전달해야 한다

J-08. 객체 정렬 — 가격 구간별 필터링 메서드 분리

문제

J-07의 `ExportCar` 클래스와 `cars` 리스트를 그대로 사용한다.

1. 사용자로부터 `minPrice`, `maxPrice`를 입력받는다 (하드코딩 허용)
2. 해당 가격 범위에 속하는 차량만 필터링하라 (경계값 포함)

3. 필터링된 결과가 없을 경우 "해당 조건의 차량이 없습니다."를 출력하라
4. 결과는 가격 오름차순으로 출력하라
5. 메서드로 분리할 것: `List<ExportCar> filterByPrice(List<ExportCar> cars, int min, int max)`

정답 코드

```
static List<ExportCar> filterByPrice(List<ExportCar> cars, int min, int max) {
    return cars.stream()
        .filter(c -> c.getPrice() >= min && c.getPrice() <= max)
        .sorted(Comparator.comparingInt(ExportCar::getPrice))
        .collect(Collectors.toList());
}

// 호출
List<ExportCar> result = filterByPrice(cars, 10000, 20000);
if (result.isEmpty()) {
    System.out.println("해당 조건의 차량이 없습니다.");
} else {
    result.forEach(System.out::println);
}
```

출제 의도

필터링 로직을 메서드로 분리하는 습관과 경계값 처리, 빈 결과 처리를 본다. 실제 서비스의 검색 필터 기능과 동일한 구조이다.

핵심 포인트

- 경계값 포함이므로 `>=`, `<=` 사용
- `result.isEmpty()` 로 빈 결과를 명시적으로 처리

함정

- `>`, `<`로 경계값을 제외하면 조건 명세와 다른 결과가 나온다
- 메서드 반환 타입을 `void`로 만들고 내부에서 출력하면 재사용이 불가능해진다

J-09. 예외 처리 — 가격 파싱 유효성 검사

문제

```
List<String> priceData = List.of("15000", "abc", "22000", "", "18500", "N/A");
```

1. 각 문자열을 정수로 변환을 시도하라
2. 변환 실패 시 (`NumberFormatException`) "잘못된 가격 데이터: [값]" 메시지를 출력하고 건너뛰어라

3. 빈 문자열은 변환 전에 별도 체크하여 "빈 값 스킵"을 출력하라
4. 유효한 가격만 모아 합계와 평균을 출력하라
5. `try-catch-finally` 구조를 사용할 것

정답 코드

```
List<Integer> validPrices = new ArrayList<>();

for (String p : priceData) {
    try {
        if (p.isEmpty()) {
            System.out.println("빈 값 스킵");
            continue;
        }
        validPrices.add(Integer.parseInt(p));
    } catch (NumberFormatException e) {
        System.out.println("잘못된 가격 데이터: [" + p + "]");
    } finally {
        // 매 항목 처리 후 공통 로직 위치 (예: 로그 기록 등)
    }
}

int sum = validPrices.stream().mapToInt(Integer::intValue).sum();
double avg =
    validPrices.stream().mapToInt(Integer::intValue).average().orElse(0);
System.out.println("합계: " + sum + ", 평균: " + avg);
```

출제 의도

`NumberFormatException` 처리와 빈 문자열 선처리 순서를 이해하는지 확인한다. 외부 데이터 수신 시 방어 코드를 작성하는 능력은 실무에서 매우 중요하다.

핵심 포인트

- 빈 문자열 체크를 `Integer.parseInt` 전에 수행해야 함
- `finally`는 정상/예외 여부 관계없이 항상 실행됨

함정

- 빈 문자열을 `parseInt`에 그냥 넘기면 `NumberFormatException`이 발생하여 별도 메시지 출력이 불가능하다
- `continue`를 `try` 블록 안에서 사용할 수 있다는 점을 모르면 구조가 복잡해진다

J-10. 예외 처리 — 커스텀 예외로 수출 불가 차량 처리

문제

1. 아래 조건에 해당하는 차량은 수출 불가로 처리한다:
 - 연식이 2010년 미만
 - 가격이 0 이하

2. `ExportException`이라는 커스텀 예외 클래스를 작성하라 (메시지 + 차량 모델명 필드 포함)
3. 아래 데이터를 순회하며 수출 불가 차량에 대해 `ExportException`을 throw하라

```
new Car("Old Model", 2008, 5000, false)
new Car("Camry", 2020, 15000, false)
new Car("Ghost Car", 2019, -1, false)
new Car("Civic", 2021, 13000, false)
```

4. `catch` 블록에서 예외 메시지를 출력하고 다음 차량으로 계속 진행하라
5. 수출 가능한 차량 목록을 최종 출력하라

정답 코드

```
class ExportException extends RuntimeException {
    private final String carModel;

    ExportException(String message, String carModel) {
        super(message);
        this.carModel = carModel;
    }

    public String getCarModel() { return carModel; }
}

// 순회 처리
List<Car> exportable = new ArrayList<>();

for (Car car : carList) {
    try {
        if (car.getYear() < 2010) {
            throw new ExportException("연식 미달 (2010년 미만)",
car.getModel());
        }
        if (car.getPrice() <= 0) {
            throw new ExportException("유효하지 않은 가격", car.getModel());
        }
        exportable.add(car);
    } catch (ExportException e) {
        System.out.println("[수출 불가] " + car.getModel() + " - " +
e.getMessage());
    }
}

System.out.println("수출 가능 차량: " + exportable.stream()
    .map(Car::getModel).collect(Collectors.toList()));
```

출제 의도

커스텀 예외 클래스 설계와 `throw/catch` 흐름 제어를 이해하는지 본다. 실무에서 도메인 예외(`BusinessException` 등)는 필수 패턴이며, 예외를 사용한 흐름 분기 능력을 검증한다.

핵심 포인트

- 커스텀 예외는 `RuntimeException` 또는 `Exception` 상속
- 예외를 `catch`한 후 `continue` 없이도 루프가 자동으로 다음 항목으로 진행됨

함정

- `throw` 후 해당 `try` 블록의 나머지 코드는 실행되지 않는다 — `exportable.add(car)`가 `throw` 뒤에 있으면 수출 불가 차량이 목록에 추가되지 않으므로 순서에 주의
- `Exception`을 상속하면 checked exception이 되어 호출부에서 `throws` 선언이 필요해진다